

Competitive Programming Notebook

py.Sandu

Contents

1	DP	2
1.1	Lis	2
1.2	Countingdp	2
1.3	Knapsack	2
2	Math	2
2.1	Factorial	2
2.2	Fastexponentiation	2
2.3	Crivo	2
2.4	Matrixexponentiation	3
2.5	Matrixmultiplication	3
2.6	Extendedeuclidean	3
3	String	3
3.1	Hash	3
4	Graph	4
4.1	Kruskal	4
4.2	Eulereanpath	4
4.3	Dijkstra	4
4.4	Bellmanford	4
4.5	Floydwarshall	5
4.6	Kahn	5
5	Geometry	5
5.1	Point	5
6	.template	6
6.1	Template	6
7	DS	6
7.1	Psum2d	6
7.2	Sparsetable	7
7.3	Segtree	7
7.4	Bit2d	7
7.5	Dsu	7
7.6	Bit	8
7.7	Seglazy	8
7.8	Monotonicstack	9

1 DP

1.1 Lis

```
1 // Find the length of the longest increasing
  subsequence in a given array
2 // Time complexity: O(n log n) where n is the length
  of the input array
```

```
3
4 int lis(vector<int>& arr) {
5     vector<int> dp;
6     for (int x : arr) {
7         auto it = lower_bound(all(dp), x);
8         if(it == dp.end()) dp.push_back(x);
9         else *it = x;
10    }
11    return dp.size();
12 }
```

1.2 Countingdp

```
1 // Counting DP (Sum of all ways / Unbounded Knapsack)
2 // Time Complexity: O(N * Target)
3 // Space Complexity: O(Target)
4 // Use case: Count number of ways to form a sum
5
6 int count_ways(vector<int>& items, int target) {
7
8     vector<int> dp(target + 1, 0);
9     dp[0] = 1;
10
11    for (int item : items) {
12        for (int i = item; i <= target; i++) {
13            dp[i] = (dp[i] + dp[i - item]) % MOD;
14        }
15    }
16
17    return dp[target];
18 }
```

1.3 Knapsack

```
1 // General take/not take dp problem
2 // Time Complexity: aprox. O(n * m)
3 // Space Complexity: O(n * m) / O(m)
4 // Use cases: maximize/minimize sum
5
6 // Top-down
7 int knapsack_top_down(vector<int>& v, vector<int>& w,
  int x){
8     int n = (int)v.size();
9     vector<vector<int>> dp(n, vector<int>(x+1, -1));
10    auto rec = [&](auto self, int i, int m) -> int {
11        if(i >= n || m == 0) return 0;
12        if(dp[i][m] != -1) return dp[i][m];
13        int take = 0, not_take = self(self, i+1, m);
14        if(m >= w[i]){
15            take = v[i] + self(self, i+1, m - w[i]);
16        }
17        return dp[i][m] = max(take, not_take);
18    };
19
20    return rec(rec, 0, x);
21 }
22
23 // Bottom-up
24 int knapsack_bottom_up(vector<int>& v, vector<int>& w,
  int x){
25     int n = (int)v.size();
26     vector<int> dp(x+1, 0);
27 }
```

```
28     for(int i=0; i < n; i++){
29         for(int m=x; m >= w[i]; m--){
30             dp[m] = max(dp[m], v[i] + dp[m - w[i]]);
31         }
32     }
33
34     return dp[x];
35 }
```

2 Math

2.1 Factorial

```
1 // Time Complexity: O(N) for precompute, O(1) for nCr
2 // Space Complexity: O(N)
3 // Use cases: calculating combinations, permutations,
  probabilities
4
5 const int MAXN, MOD;
6 vector<int> fact(MAXN), ifact(MAXN);
7
8 void precompute(){
9
10    fact[0] = 1; ifact[0] = 1;
11    for(int i=1; i<MAXN; i++){
12        fact[i] = (i * fact[i-1]) % MOD;
13    }
14
15    ifact[MAXN-1] = fexp(fact[MAXN-1], MOD-2, MOD);
16    for(int i=MAXN-2; i >= 1; i--){
17        ifact[i] = (ifact[i+1]*(i+1)) % MOD;
18    }
19 }
20
21 int choose(int n, int r){
22     if(r < 0 or r > n) return 0;
23     int num = fact[n];
24     int den = (ifact[r] * ifact[n - r]) % MOD;
25     return (num * den) % MOD;
26 }
```

2.2 Fastexponentiation

```
1 // Calculates (x^n) % MOD
2 // Time Complexity: O(log n)
3 // Space Complexity: O(1)
4
5 int fexp(int x, int b, int MOD){
6     int res = 1;
7
8     while(b > 0){
9         if(b & 1) res = (res * x) % MOD;
10        x = (x * x) % MOD;
11        b >>= 1;
12    }
13    return res;
14 }
```

2.3 Crivo

```
1 // Find all prime numbers up to a given limit using
  the Sieve of Eratosthenes algorithm
2 // Time complexity: O(n log log n) where n is the
  limit
3 // This idea can also be used to construct a table
  for the smallest prime factors, O(log N) per
  query
4
5 vector<int> findPrimes(int limit) {
6     vector<bool> isPrime(limit + 1, true);
7     isPrime[0] = isPrime[1] = false;
```

```

8
9   for (int i = 2; i * i <= limit; i++) {
10       if (isPrime[i]) {
11           for (int j = i * i; j <= limit; j += i) {
12               isPrime[j] = false;
13           }
14       }
15   }
16
17   vector<int> primes;
18   for (int i = 2; i <= limit; i++) {
19       if (isPrime[i]) {
20           primes.push_back(i);
21       }
22   }
23   return primes;
24 }
25
26 vector<int> buildSPF(int limit){
27     vector<int> spf(limit+1);
28     iota(spf.begin(), spf.end(), 0);
29     for(int i = 2; i * i <= limit; i++) {
30         if(spf[i] == i){
31             for(int j = i * i; j <= limit; j += i) {
32                 if(spf[j] == j) spf[j] = i;
33             }
34         }
35     }
36
37     return spf;
38 }
39
40 vector<int> getFactors(vector<int>& spf, int x){
41     vector<int> factors;
42     while(x > 1){
43         factors.push_back(spf[x]);
44         x /= spf[x];
45     }
46
47     return factors;
48 }

```

2.4 Matrixexponentiation

```

1 // Calculate the Nth Fibonacci number using matrix
  exponentiation.
2 // Time Complexity: O(log N)
3 // Space Complexity: O(1)
4
5 void matrixExponentiation(vector<vector<long long>>&
  matrix, int power) {
6     int n = matrix.size();
7     vector<vector<long long>> result(n, vector<long
  long>(n, 0));
8
9     // Initialize result as the identity matrix
10    for (int i = 0; i < n; i++) {
11        result[i][i] = 1;
12    }
13
14    while (power) {
15        if (power % 2 == 1) {
16            matrixMultiplication(result, matrix,
  result);
17        }
18        matrixMultiplication(matrix, matrix, matrix);
19        power /= 2;
20    }
21
22    matrix = result;
23 }

```

2.5 Matrixmultiplication

```

1 // Multiply two matrices A and B, and return the
  result in matrix C.
2 // Time Complexity: O(n^3)
3 // Space Complexity: O(n^2)
4
5 vector<vector<int>> matrixMultiplication(const vector
  <vector<int>>& A, const vector<vector<int>>& B) {
6     int n = A.size();
7     vector<vector<int>> C(n, vector<int>(n, 0)); //
  Initialize result matrix with zeros
8
9     for (int i = 0; i < n; i++) {
10        for (int j = 0; j < n; j++) {
11            for (int k = 0; k < n; k++) {
12                C[i][j] += A[i][k] * B[k][j];
13            }
14        }
15    }
16    return C;
17 }

```

2.6 Extendedeuclidean

```

1 // Extended Euclidean Algorithm
2 // Returns (gcd(a, b), x, y) such that a*x + b*y =
  gcd(a, b)
3
4 int euclides(int a, int b, int &x, int &y){
5     if(b == 0){
6         x = 1, y = 0;
7         return a;
8     }
9     int x1, y1;
10    int g = euclides(b, a%b, x1, y1);
11    x = y1; y = x1 - y1*(a/b);
12    return g;
13 }
14
15 int inverso_mod(int a, int m){
16     int x, y;
17     int g = euclides(a, m, x, y);
18     if(g != -1) return -1;
19     return (x%m + m) % m;
20 }

```

3 String

3.1 Hash

```

1 // String Hash template
2 // constructor(s) - O(|s|)
3 // query(l, r) - returns the hash of the range [l,r]
  from left to right - O(1)
4 // query_inv(l, r) from right to left - O(1)
5 // patrocinado por tiagodfs
6
7 mt19937 rng(time(nullptr));
8
9 struct Hash {
10     const int X = rng();
11     const int MOD = 1e9+7;
12     int n; string s;
13     vector<int> h, hi, p;
14     Hash() {}
15     Hash(string s): s(s), n(s.size()), h(n), hi(n), p
  (n) {
16         for (int i=0;i<n;i++) p[i] = (i ? X*p[i-1]:1)
  % MOD;
17         for (int i=0;i<n;i++)

```

```

18         h[i] = (s[i] + (i ? h[i-1]:0) * X) % MOD;
19         for (int i=n-1;i>=0;i--)
20             hi[i] = (s[i] + (i+1<n ? hi[i+1]:0) * X)
21             % MOD;
22     }
23     int query(int l, int r) {
24         int hash = (h[r] - (l ? h[l-1]*p[r-l+1]%MOD :
25         0));
26         return hash < 0 ? hash + MOD : hash;
27     }
28     int query_inv(int l, int r) {
29         int hash = (hi[l] - (r+1 < n ? hi[r+1]*p[r-l
30         +1] % MOD : 0));
31         return hash < 0 ? hash + MOD : hash;
32     }
33 };

```

4 Graph

4.1 Kruskal

```

1 // Kruskal algorithm for finding the Minimum Spanning
2 // Tree
3 // Time Complexity: O(E log E)
4 // Space complexity: O(V + E)
5 // Use cases: connect nodes with min/max cost,
6 // clustering, minimax path
7
8 // edges: {weight, u, v}
9 vector<vector<pair<int, int>>> kruskalMST(vector<
10 array<int, 3>>& edges, int n){
11     int m = (int)edges.size();
12     sort(edges.begin(), edges.end());
13     vector<vector<pair<int, int>>> mst(n+1);
14     DSU dsu(n+1); int cost = 0;
15     for(int i=0; i<m; i++){
16         int w = edges[i][0], a = edges[i][1], b =
17         edges[i][2];
18         if(dsu.merge(a, b)){
19             mst[a].push_back({w, b});
20             mst[b].push_back({w, a});
21             cost += w;
22         }
23     }
24     return mst;
25 }

```

4.2 Eulereanpath

```

1 // Find the Eulerian path in a graph using Hierholzer
2 // 's algorithm
3 // Time complexity: O(E) where E is the number of
4 // edges in the graph
5 // Use cases: visit every edge once, DNA sequencing,
6 // puzzles, drawing figures
7
8 vector<int> findEulerianPath(vector<vector<int>>& adj
9 , int start) {
10
11     vector<int> path;
12     stack<int> s;
13     s.push(start);
14
15     while (!s.empty()) {
16         int i = s.top();
17         if (adj[i].empty()) {
18             path.push_back(i);
19             s.pop();
20         } else {
21             int u = adj[i].back(); adj[i].pop_back();
22             s.push(u);
23         }
24     }
25 }

```

```

18     }
19     }
20     }
21     }
22     reverse(path.begin(), path.end());
23     return path;
24 }

```

4.3 Dijkstra

```

1 // Find shortest path from a source vertex to all
2 // other vertices in a graph with non-negative edge
3 // weights
4 // Time Complexity: O((V + E) log V)
5 // Space Complexity: O(V)
6 // Use cases: shortest path in non-negative weighted
7 // graphs
8
9 vector<int> dijkstra(vector<vector<pair<int, int>>>&
10 adj, int src) {
11
12     int n = adj.size() - 1;
13     vector<int> dist(n+1, INF);
14     dist[src] = 0;
15
16     priority_queue<pair<int, int>, vector<pair<int,
17     int>>, greater<pair<int, int>>> pq; // Min-heap
18     priority queue
19     pq.push({0, src});
20
21     while (!pq.empty()) {
22         auto [d, i] = pq.top();
23         pq.pop();
24
25         if(d > dist[i]) continue;
26
27         for (auto [u, w] : adj[i]) {
28             if (dist[i] + w < dist[u]) {
29                 dist[u] = dist[i] + w;
30                 pq.push({dist[u], u});
31             }
32         }
33     }
34
35     return dist;
36 }

```

4.4 Bellmanford

```

1 // Find the shortest path from a source vertex to all
2 // other vertices in a graph with negative weight
3 // edges and detect negative weight cycles.
4 // Time Complexity: O(V * E)
5 // Space Complexity: O(V)
6 // Use cases: SSSP with negative edges, negative
7 // cycle detection, system of difference constraints
8 // , shortest path with at most K edges
9
10 bool bellmanFord(vector<vector<pair<int, int>>>& adj,
11 int src) {
12
13     int n = adj.size() - 1;
14     vector<int> dist(n+1, INF);
15     dist[src] = 0;
16
17     // Relax all edges n-1 times
18     for (int i = 1; i < n; i++) {
19         for (int u = 1; u <= n; u++) {
20             for (auto& [v, w] : adj[u]) {
21                 if (dist[u] != INF and dist[u] + w <
22                 dist[v]) {
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99

```

```

17         dist[v] = dist[u] + w;
18     }
19 }
20 }
21 }
22
23 // Check for negative weight cycles
24 for (int u = 1; u <= n; u++) {
25     for (auto& [v, w] : adj[u]) {
26         if (dist[u] != INF and dist[u] + w < dist
27 [v]) {
28             // Graph contains negative weight
29             cycle
30             return true;
31         }
32     }
33 }
34 return false;
35 }

```

4.5 Floydwarshall

```

1 // Find the shortest paths in a weighted graph with
2 // positive or negative edge weights (but with no
3 // negative cycles).
4 // Time Complexity: O(V^3)
5 // Space Complexity: O(V^2)
6 // Use cases: reachability, minimax distance, global
7 // negative cycle detection(dist[i][i] < 0)
8
9 void floydWarshall(vector<vector<int>>& dist) { //
10     dist(INF)
11
12     int n = dist.size() - 1;
13     for(int i = 1; i <= n; i++) dist[i][i] = 0;
14
15     for (int k = 1; k <= n; k++) {
16         for (int i = 1; i <= n; i++) {
17             for (int j = 1; j <= n; j++) {
18                 if (dist[i][k] != INF and dist[k][j]
19 != INF) {
20                     dist[i][j] = min(dist[i][j], dist
21 [i][k] + dist[k][j]);
22                 }
23             }
24         }
25     }
26 }

```

4.6 Kahn

```

1 // Topological Sort (Kahn's Algorithm)
2 // Time Complexity: O(V + E)
3 // Space Complexity: O(V)
4 // Use cases: Dependency resolution, cycle detection
5 // in directed graphs, DP on DAGs
6
7 vector<int> TopologicalSort(vector<vector<int>>& adj,
8 int n){
9     vector<int> degree(n+1, 0);
10    for(int i=1; i<=n; i++){
11        for(auto u: adj[i]) degree[u]++;
12    }
13
14    // priority_queue in case of lexicographically
15    // smallest answer
16    queue<int> q; vector<int> ans;
17    for(int i=1; i<=n; i++){
18        if(degree[i] == 0){
19            q.push(i); ans.push_back(i);
20        }
21    }

```

```

18     }
19 }
20 }
21 while(!q.empty()){
22     int i = q.front(); q.pop();
23     for(auto u: adj[i]){
24         degree[u]--;
25         if(degree[u] == 0){
26             q.push(u); ans.push_back(u);
27         }
28     }
29 }
30
31 if((int)ans.size() != n) return {};
32 return ans;
33 }
34 }

```

5 Geometry

5.1 Point

```

1
2 const double EPS = 1e-9;
3 // EPS = 0 -> int
4
5 struct Point {
6     double x, y;
7
8     Point() : x(0), y(0) {}
9     Point(double x, double y) : x(x), y(y) {}
10
11     Point operator+(const Point& p) const { return
12     Point(x + p.x, y + p.y); }
13     Point operator-(const Point& p) const { return
14     Point(x - p.x, y - p.y); }
15     Point operator*(double c) const { return Point(x
16     * c, y * c); }
17     Point operator/(double c) const { return Point(x
18     / c, y / c); }
19
20     bool operator<(const Point& p) const {
21         if (abs(x - p.x) > EPS) return x < p.x;
22         return y < p.y - EPS;
23     }
24
25     bool operator==(const Point& p) const {
26         return abs(x - p.x) <= EPS && abs(y - p.y) <=
27         EPS;
28     }
29 };
30
31 // > 0: B esq. vetor OA, < 0: B dir. vetor OA, 0:
32 // Colineares
33 double cross_product(Point O, Point A, Point B) {
34     return (A.x - O.x) * (B.y - O.y) - (A.y - O.y) *
35     (B.x - O.x);
36 }
37
38 // distancia euclidiana
39 double dist(Point A, Point B) {
40     return hypot(A.x - B.x, A.y - B.y);
41 }
42
43 // distancia quadrada
44 double dist2(Point A, Point B) {
45     return (A.x - B.x) * (A.x - B.x) + (A.y - B.y) *
46     (A.y - B.y);
47 }
48
49 // produto escalar, dot == 0 perpendiculares
50 double dot_product(Point O, Point A, Point B) {

```

```

43     return (A.x - O.x) * (B.x - O.x) + (A.y - O.y) *
      (B.y - O.y);
44 }
45
46 int get_half(Point O, Point P) {
47     if (P.y - O.y > EPS or (abs(P.y - O.y) <= EPS and
      P.x - O.x > EPS)) return 1;
48     return 0;
49 }
50
51 // ordenacao por perimetro
52 void sort_by_angle(vector<Point>& p, Point center) {
53     sort(p.begin(), p.end(), [&](const Point& A,
      const Point& B) {
54         int halfA = get_half(center, A);
55         int halfB = get_half(center, B);
56
57         if (halfA != halfB) {
58             return halfA > halfB;
59         }
60
61         double cross = cross_product(center, A, B);
62         if (abs(cross) > EPS) {
63             return cross > 0;
64         }
65
66         return dist2(center, A) < dist2(center, B) -
      EPS;
67     });
68 }
69
70 // shoelace formula, vÃrtices do polÃgono ordenados
      pelo perÃmetro.
71 double polygon_area(const vector<Point>& p) {
72     double area = 0.0;
73     int n = p.size();
74
75     for (int i = 0; i < n; i++) {
76         int next_i = (i + 1) % n;
77         area += (p[i].x * p[next_i].y) - (p[next_i].x
      * p[i].y);
78     }
79
80     // return abs(area) -> int
81     return abs(area) / 2.0;
82 }
83
84 // aux func
85 int orientation(Point O, Point A, Point B) {
86     double val = cross_product(O, A, B);
87     if (abs(val) <= EPS) return 0; // Colinear
88     return (val > 0) ? 1 : -1;    // 1: Esquerda,
      -1: Direita
89 }
90
91 // 2. P in seg AB?
92 bool on_segment(Point P, Point A, Point B) {
93     return P.x >= min(A.x, B.x) - EPS && P.x <= max(A
      .x, B.x) + EPS &&
94     P.y >= min(A.y, B.y) - EPS && P.y <= max(A
      .y, B.y) + EPS;
95 }
96
97 // Ab seg cross CD?
98 bool segments_intersect(Point A, Point B, Point C,
      Point D) {
99     // Calcula as 4 orientaÃÃes necessÃrias
100     int o1 = orientation(A, B, C);
101     int o2 = orientation(A, B, D);
102     int o3 = orientation(C, D, A);
103     int o4 = orientation(C, D, B);
104
105     if (o1 != o2 && o3 != o4) {

```

```

        return true;
    }
    if (o1 == 0 && on_segment(C, A, B)) return true;
    // C toca AB
    if (o2 == 0 && on_segment(D, A, B)) return true;
    // D toca AB
    if (o3 == 0 && on_segment(A, C, D)) return true;
    // A toca CD
    if (o4 == 0 && on_segment(B, C, D)) return true;
    // B toca CD
    return false; // NÃo se cruzam
}

```

6 .template

6.1 Template

```

1 #include<bits/stdc++.h>
2
3 using namespace std;
4
5 #define int long long
6 #define vi vector<int>
7 #define pii pair<int, int>
8 #define all(x) (x).begin(), (x).end()
9 #define debug(x...) cout<<"[#x]": ",[]{(auto...$)}{((
      cout<<$<<" "; ...);} (x),cout<<endl
10
11 void solve(){
12
13 }
14
15 signed main(){
16     ios::sync_with_stdio(false);
17     cin.tie(NULL);
18     int t = 1;
19     //cin >> t;
20     while(t--) solve();
21     return 0;
22 }

```

7 DS

7.1 Psum2d

```

1 // Psum2D
2 // Time Complexity: Query O(1)
3 // Space Complexity: O(N*M)
4 // Use cases: static 2d range sum, fixed-size window
      search, max sum submatrix
5
6 int n, m;
7 vector<vector<int>> arr(n+1, vector<int>(m+1, 0));
8 vector<vector<int>> psum(n+1, vector<int>(m+1, 0));
9
10 for (int i = 1; i <= n; i++) {
11     for (int j = 1; j <= m; j++) {
12         cin >> arr[i][j];
13         psum[i][j] = arr[i][j]
14             + psum[i][j-1]
15             + psum[i-1][j]
16             - psum[i-1][j-1];
17     }
18 }
19
20 // canto superior esquerdo (x1, y1) canto inferior
      direito (x2, y2)
21 int x1, y1, x2, y2;

```

```

22 int res = psum[x2][y2]
23         - psum[x1 - 1][y2]
24         - psum[x2][y1 - 1]
25         + psum[x1 - 1][y1 - 1];

```

7.2 Sparsetable

```

1 // Sparse Table
2 // Time Complexity: O(n * log n) build, O(1) query
3 // Space complexity: O(n * log n)
4 // Idempotent operation f(a, a) = a (min, max, gcd,
5 // Use cases: idempotent functions range queries
6 // and, or)
7 struct SparseTable{
8
9     int n, p;
10    vector<vector<int>> st;
11
12    SparseTable(vector<int> &arr){
13        n = (int) arr.size();
14        p = __lg(n);
15        st.assign(p+1, vector<int>(n, 0));
16
17        for(int j=0; j<n; j++) st[0][j] = arr[j];
18        for(int i = 1; i <= p; i++){
19            int k = (1ll << (i-1));
20            int lim = n - 2*k + 1;
21            for(int j=0; j<lim; j++){
22                st[i][j] = op(st[i-1][j], st[i-1][j+k]);
23            }
24        }
25    }
26
27    int query(int l, int r){
28        int len = r - l + 1;
29        int p_local = __lg(len), k = (1ll << p_local);
30        return op(st[p_local][l], st[p_local][r - k + 1]);
31    }
32 }
33 };

```

7.3 Segtree

```

1 template <typename T>
2
3 struct SegTree{
4     int n;
5     vector<T> tree;
6     const T NEUTRO = 1e18;
7
8     T merge(T a, T b) {return min(a, b);}
9     SegTree(int n){
10         this->n = n;
11         tree.assign(2*n, NEUTRO);
12     }
13
14     void build(vector<T>& v){
15         for(int i=0; i<n; i++) tree[n+i] = v[i];
16         for(int i=n-1; i>0; i--) tree[i] = merge(tree[2*i], tree[2*i + 1]);
17     }
18
19     void update(int pos, T val){
20         pos += n;
21         tree[pos] = val;
22         for(pos /= 2; pos > 0; pos /= 2) tree[pos] = merge(tree[pos*2], tree[pos*2 + 1]);
23     }

```

```

24
25 T query(int l, int r){
26     T res_l = NEUTRO, res_r = NEUTRO;
27     for(l += n, r += n; l <= r; l /= 2, r /= 2){
28         if(l%2 != 0) res_l = merge(res_l, tree[l++]);
29         if(r%2 == 0) res_r = merge(tree[r--], res_r);
30     }
31     return merge(res_l, res_r);
32 }
33
34 };

```

7.4 Bit2d

```

1 // Binary Indexed Tree(BIT) 2D
2 // Time complexity: query and update log N * log M
3 // Space complexity: O(N*M)
4 // Use cases: dynamic 2D prefix sums, counting active
5 // points in a dynamic rectangle
6 struct BIT2D {
7
8     int n, m; vector<vector<int>> bit;
9     BIT2D(int n, int m) : n(n), m(m) {
10         bit.assign(n+1, vector<int>(m+1, 0));
11     }
12
13     void update(int x, int y, int val) {
14         for (int i = x; i <= n; i += i & -i) {
15             for (int j = y; j <= m; j += j & -j) {
16                 bit[i][j] += val;
17             }
18         }
19     }
20
21     int query(int x, int y) {
22         int sum = 0;
23         for (int i = x; i > 0; i -= i & -i) {
24             for (int j = y; j > 0; j -= j & -j) {
25                 sum += bit[i][j];
26             }
27         }
28         return sum;
29     }
30
31     int queryRange(int x1, int y1, int x2, int y2) {
32         return query(x2, y2)
33             - query(x1 - 1, y2)
34             - query(x2, y1 - 1)
35             + query(x1 - 1, y1 - 1);
36     }
37 };
38 };

```

7.5 Dsu

```

1 // Disjoint Set Union(DSU)
2 // Time Complexity: aprox. O(1)
3 // Space Complexity: O(n)
4 // Use cases: connected components, cycle detection,
5 // offline edge deletion
6 struct DSU{
7
8     vector<int> parent, size;
9
10    DSU(int n){
11        parent.assign(n+1, 0);
12        size.assign(n+1, 1);
13        iota(parent.begin(), parent.end(), 0);

```

```

14     }
15
16     int find(int v){
17         if(v == parent[v]) return v;
18         return parent[v] = find(parent[v]);
19     }
20
21     bool same(int a, int b){
22         return find(a) == find(b);
23     }
24
25     bool merge(int a, int b){
26         a = find(a);
27         b = find(b);
28         if(a != b){
29             if(size[a] < size[b]) swap(a, b);
30             parent[b] = a;
31             size[a] += size[b];
32             return true;
33         }
34         return false;
35     }
36
37 };

```

7.6 Bit

```

1 // Binary Indexed Tree (BIT)
2 // Time complexity: query and update log N
3 // Space complexity: O(N)
4 // Use cases: dynamic psum, inversion counting,
5 // dynamic multiset
6
7 struct BIT{
8
9     int n; vector<int> bit;
10     BIT(int n) : n(n){
11         bit.assign(n+1, 0);
12     }
13
14     void update(int i, int x){
15         while(i <= n){
16             bit[i] += x;
17             i += i & -i;
18         }
19     }
20
21     int query(int i){
22         int sum = 0;
23         while(i > 0){
24             sum += bit[i];
25             i -= i & -i;
26         }
27         return sum;
28     }
29
30     int queryRange(int l, int r){
31         return query(r) - query(l-1);
32     }
33
34 // retorna o Índice do k-ésimo elemento da bit
35 int find_kth(int k){
36
37     int idx = 0;
38     int max_bit = __lg(n);
39
40     for(int i = max_bit; i >= 0; i--){
41         int next_idx = idx + (1ll << i);
42         if(next_idx <= n and bit[next_idx] < k){
43             idx = next_idx;
44             k -= bit[next_idx];
45         }
46     }
47 }

```

```

46
47         return idx+1;
48     }
49
50 };

```

7.7 Seglazy

```

1 template <typename T>
2
3 struct SegLazy{
4     int n;
5     vector<T> tree, lazy;
6     const T NEUTRO = 0, LAZY_NEUTRO = 0;
7
8     T merge(T a, T b) {return a + b;}
9
10    SegLazy(int n){
11        this->n = n;
12        tree.assign(4*n, NEUTRO);
13        lazy.assign(4*n, NEUTRO);
14    }
15
16    void push(int node, int l, int r){
17        if(lazy[node] == LAZY_NEUTRO) return;
18        tree[node] += lazy[node] * (r - l + 1);
19        if(l != r){
20            lazy[2*node] += lazy[node];
21            lazy[2*node + 1] += lazy[node];
22        }
23        lazy[node] = LAZY_NEUTRO;
24    }
25
26    void build(int node, int l, int r, vector<T> &v){
27        if(l == r){
28            tree[node] = v[l]; return;
29        }
30        int mid = (l+r)/2;
31        build(2*node, l, mid, v);
32        build(2*node + 1, mid+1, r, v);
33        tree[node] = merge(tree[2*node], tree[2*node
34        + 1]);
35    }
36
37    void update(int node, int l, int r, int ql, int
38    qr, T val){
39        push(node, l, r);
40        if(ql > r or qr < l) return;
41        if(ql <= l and r <= qr){
42            lazy[node] += val;
43            push(node, l, r);
44            return;
45        }
46        int mid = (l+r)/2;
47        update(2*node, l, mid, ql, qr, val);
48        update(2*node + 1, mid+1, r, ql, qr, val);
49        tree[node] = merge(tree[2*node], tree[2*node
50        + 1]);
51    }
52
53    T query(int node, int l, int r, int ql, int qr){
54        push(node, l, r);
55        if(ql > r or qr < l) return NEUTRO;
56        if(ql <= l and r <= qr) return tree[node];
57        int mid = (l+r)/2;
58        return merge(query(2*node, l, mid, ql, qr),
59        query(2*node + 1, mid + 1, r, ql, qr));
60    }
61
62    void build(const vector<T> &v) { build(1, 0, n -
63    1, v); }
64    void update(int ql, int qr, T val) { update(1, 0,
65    n - 1, ql, qr, val); }

```



```

60     T query(int ql, int qr) { return query(1, 0, n - 1, ql, qr); }
61
62
63 };

```

7.8 Monotonicstack

```

1 // Monotonic Stack, next greater number in a circular array
2 // Time Complexity: O(N)
3 // Space Complexity: O(N)
4 // Use cases: ring problems, finding next larger obstacle in a loop
5
6 for (int i = 0; i < 2 * n; i++) {
7     int num = a[i % n];
8     while (!st.empty() and a[st.top()] < num) { // remove os menores
9         res[st.top()] = num; st.pop();
10    }
11
12    if (i < n) st.push(i);
13 }
14
15 // Monotonic Stack, next smaller element in a linear array
16 // Use cases: largest rectangle in histogram, subarray contribution
17
18 for (int i = 0; i < n; i++) {
19     while (!st.empty() and a[st.top()] > a[i]) { // remove os maiores
20         res[st.top()] = a[i];
21         st.pop();
22     }
23     st.push(i);
24 }

```